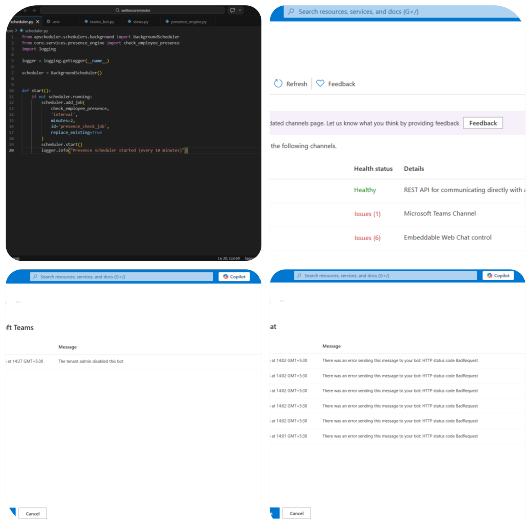

okay today you have to help me with veri precision
i am creating a messaging bot for microsoft teams which will send reminders like drink water strech yourself and like that so now i am using django backend in which my backend code lies
i have shared folder structure to you
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog
from core.services.graph import get_user_presence

```
WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting"
```

```
]


def check_employee_presence():

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )

    for employee in employees:

        try:
            presence_data = get_user_presence(employee.azure_user_id)
            status = presence_data.get("availability")

            print(f"{employee.display_name} status: {status}")
        except Exception as e:
            print(f"Presence fetch failed: {e}")
            continue

        # Save presence log
        PresenceLog.objects.create(
            employee=employee,
            status=status
        )

        tracker, _ = ActivityTracker.objects.get_or_create(
            employee=employee
```

```
        )

        if status in WORKING_STATUSES:

            tracker.consecutive_available_count += 1
            print(f"Working detected ->
{tracker.consecutive_available_count}")

        else:

            print("Resetting counter")

            tracker.consecutive_available_count = 0
            # tracker.reminder_sent = False


        if tracker.consecutive_available_count >= 6:

            ReminderLog.objects.create(
                employee=employee,
                message="Drink water and stretch yourself!"
            )

            print(f"Reminder triggered for {employee.display_name}")

            tracker.consecutive_available_count = 0

        tracker.last_status = status
        tracker.save()
```

```
this is presense_engine.py
import requests
import msal
from django.conf import settings

AUTHORITY =
f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
SCOPE = ["https://graph.microsoft.com/.default"]

_app = msal.ConfidentialClientApplication(
    settings.GRAPH_CLIENT_ID,
    authority=AUTHORITY,
    client_credential=settings.GRAPH_CLIENT_SECRET,
)


def get_access_token():
    """
    Always acquire token using MSAL.
    MSAL automatically caches and refreshes tokens.
    """

    result = _app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        return result["access_token"]

    raise Exception(f"Token error: {result}")
```

```
def get_user_presence(user_id):
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()

    raise Exception(
        f"Graph Error {response.status_code}: {response.text}"
    )
```

this is graph.py

```
import os
from dotenv import load_dotenv
from botbuilder.core import BotFrameworkAdapterSettings,
BotFrameworkAdapter

load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()
```

```
print("BOT APP ID:", BOT_APP_ID)
print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else
"NO")
print("BOT PASSWORD LENGTH:", len(BOT_APP_PASSWORD) if
BOT_APP_PASSWORD else 0)

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD
)

ADAPTER = BotFrameworkAdapter(SETTINGS)
```

teamsbot.py

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog

admin.site.register(Employee)
admin.site.register(PresenceLog)
admin.site.register(ActivityTracker)
admin.site.register(ReminderLog)
```

this is admin.py

```
import uuid
from django.db import models

class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
```

```
    azure_user_id = models.CharField(max_length=255, unique=True)
    display_name = models.CharField(max_length=255)
    email = models.EmailField()
    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)
    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name


class PresenceLog(models.Model):
    employee = models.ForeignKey(Employee,
on_delete=models.CASCADE)
    status = models.CharField(max_length=50)
    checked_at = models.DateTimeField(auto_now_add=True)


class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee,
on_delete=models.CASCADE)
    consecutive_available_count = models.IntegerField(default=0)
    last_status = models.CharField(max_length=50, null=True,
blank=True)
    reminder_sent = models.BooleanField(default=False)


class ReminderLog(models.Model):
    employee = models.ForeignKey(Employee,
```

```
on_delete=models.CASCADE)
    sent_at = models.DateTimeField(auto_now_add=True)
    message = models.TextField()
```

models.py

```
from apscheduler.schedulers.background import BackgroundScheduler
from core.services.presence_engine import check_employee_presence
import logging

logger = logging.getLogger(__name__)

scheduler = BackgroundScheduler()



def start():
    if not scheduler.running:
        scheduler.add_job(
            check_employee_presence,
            'interval',
            minutes=2,
            id='presence_check_job',
            replace_existing=True
        )
        scheduler.start()
        logger.info("Presence scheduler started (every 10 minutes)")
```

scheduler.py

```
from rest_framework import serializers
from core.models import Employee, PresenceLog, ReminderLog
```

```
class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = "__all__"


class PresenceLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = PresenceLog
        fields = "__all__"


class ReminderLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = ReminderLog
        fields = "__all__"
```

serializers.py

```
from django.urls import path
from core.views import (
    RunPresenceCheck,
    TestPresence,
    EmployeeListCreate,
    bot_messages,
)

urlpatterns = [
    path("employees/", EmployeeListCreate.as_view()),
    path("test-presence/", TestPresence.as_view()),
    path("run-check/", RunPresenceCheck.as_view()),
```

```
    path("bot/messages/", bot_messages),
]
urls.py
import json
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from core.models import Employee
from core.serializers import EmployeeSerializer
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

from botbuilder.schema import Activity
from botbuilder.core import TurnContext
from .services.teams_bot import ADAPTER


class EmployeeListCreate(APIView):

    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)
```

```
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
status=status.HTTP_201_CREATED)

        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)


class TestPresence(APIView):

    def get(self, request):

        employee = Employee.objects.first()

        if not employee:
            return Response({"error": "No employee found"})

        data = get_user_presence(employee.azure_user_id)

        return Response(data)


class RunPresenceCheck(APIView):

    def post(self, request):

        check_employee_presence()
```

```
        return Response({"message": "Presence check executed"})


@csrf_exempt
async def bot_messages(request):

    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"})

    body = json.loads(request.body)

    activity = Activity().deserialize(body)

    auth_header = request.headers.get("Authorization", "")

    async def call_bot_logic(turn_context: TurnContext):

        user_text = turn_context.activity.text

        user_id = turn_context.activity.from_property.id
        user_name = turn_context.activity.from_property.name
        conversation_id = turn_context.activity.conversation.id
        tenant_id = turn_context.activity.conversation.tenant_id

        print("\n------ Teams Message Received ------")
        print("User Name:", user_name)
        print("User ID:", user_id)
        print("Conversation ID:", conversation_id)
```

```
        print("Tenant ID:", tenant_id)
        print("Message:", user_text)
        print("-----------------------------------\n")

        if user_text:

            reply = f"Hello {user_name} 👋 I received: {user_text}"

            await turn_context.send_activity(reply)

    await ADAPTER.process_activity(
        activity,
        auth_header,
        call_bot_logic
    )

    return JsonResponse({"status": "ok"})
```

views.py

```
"""
Django settings for wellnessreminder project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""
```

```
import os
from pathlib import Path
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent

load_dotenv(BASE_DIR / ".env")

DEBUG = os.getenv("DEBUG") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See
https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
f@a+&a%uw2)h(%g+z8a93q5c@6_y6q!52k8eg1gyzs$akwkx))'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True
```

```
ALLOWED_HOSTS = [
    "127.0.0.1",
    "localhost",
    "unsecured-elsewhere-lea.ngrok-free.dev"
]


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'corsheaders',
    'core.apps.CoreConfig',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
```

```
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'wellnessreminder.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'wellnessreminder.wsgi.application'


# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
```

```
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}


# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

```
# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'Asia/Kolkata'

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
CORS_ALLOW_ALL_ORIGINS = True

CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
```

```
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    "check-presence-every-10-minutes": {
        "task": "core.tasks.run_presence_check_task",
        "schedule": crontab(minute="*/10"),
    },
}
settings.py
aiohappyeyeballs==2.6.1
aiohttp==3.13.3
aiosignal==1.4.0
amqp==5.3.1
aniso8601==7.0.0
APScheduler==3.11.2
asgiref==3.11.1
attrs==25.4.0
autobahn==25.10.2
Automat==25.4.16
azure-core==1.38.2
billiard==4.2.4
botbuilder-core==4.17.1
botbuilder-schema==4.17.1
botframework-connector==4.17.1
botframework-streaming==4.17.1
celery==5.6.2
certifi==2026.2.25
cffi==2.0.0
```

```
channels==4.3.2
charset-normalizer==3.4.4
click==8.3.1
click-didyoumean==0.3.1
click-plugins==1.1.1.2
click-repl==0.3.0
colorama==0.4.6
constantly==23.10.4
cron_descriptor==2.0.6
cryptography==46.0.5
daphne==4.2.1
diff-match-patch==20241021
Django==5.0
django-allauth==65.14.3
django-appconf==1.2.0
django-avatar==9.0.0
django-celery-beat==2.8.1
django-channels-graphql-ws==0.5.0
django-cors-headers==4.9.0
django-extensions==4.1
django-filter==25.1
django-graphql-auth==0.3.7
django-graphql-jwt==0.4.0
django-import-export==4.4.0
django-markdownx==4.0.9
django-oauth-toolkit==3.2.0
django-timezone-field==7.2.1
django-utils==0.0.2
django_celery_results==2.6.0
```

```
djangorestframework==3.16.1
djangorestframework_simplejwt==5.5.1
dnspython==2.8.0
dotenv==0.9.9
frozenlist==1.8.0
graphene==2.1.9
graphene-django==2.16.0
graphene-file-upload==1.3.0
graphql-core==2.3.2
graphql-relay==2.0.1
hyperlink==21.0.0
idna==3.11
Incremental==24.11.0
isodate==0.7.2
jsonpickle==3.4.2
jwcrypto==1.5.6
kombu==5.6.2
Markdown==3.10.2
msal==1.35.0
msgpack==0.6.2
msrest==0.7.1
multidict==6.7.1
oauthlib==3.3.1
packaging==26.0
pillow==12.1.1
promise==2.3
prompt_toolkit==3.0.52
propcache==0.4.1
pyasn1==0.6.2
```

```
pyasn1_modules==0.4.2
pycparser==3.0
PyJWT==2.11.0
PyMySQL==1.1.2
pyOpenSSL==25.3.0
python-crontab==3.3.0
python-dateutil==2.9.0.post0
python-dotenv==1.2.1
python-magic-bin==0.4.14
pytz==2025.2
qrcode==8.2
redis==7.2.1
requests==2.32.5
requests-oauthlib==2.0.0
Rx==1.6.3
service-identity==24.2.0
simpleeval==1.0.3
singledispatch==4.1.2
six==1.17.0
sqlparse==0.5.5
tablib==3.9.0
text-unidecode==1.3
Twisted==25.5.0
txaio==25.12.2
typing_extensions==4.15.0
tzdata==2025.3
tzlocal==5.3.1
urllib3==2.6.3
vine==5.1.0
```

wcwidth==0.6.0
yarl==1.22.0
zope.interface==8.2
requirements.txt
now what i am planning is i fetch teams employee presense and store it if employee is working straight for 1 hour a reminder is triggered for that employee to drink water or whatever message i put i am using ngrok for local development, my app registered in azure i added app bot id/app id tenant id and secret in .env file also i created wellness reminder bot in azure resource group and bot is secret tenant id is also added in .env correctlly bot is installed in microsoft teams but when i run backend locally and turning on ngrok to get temporary url and message anything on that bot it receives the chat but unable to send me back any message also when a reminder trigger is generated i want it to send message to employee for which reminder is triggered
"Sangram status: Available
Working detected -> 5
Senthil status: Away
Sangram status: Available
Resetting counter
Working detected -> 6
Reminder triggered for Sangram
sejal status: Available
Working detected -> 1
Senthil status: Away
Resetting counter
sejal status: Available
Working detected -> 2
Sangram status: Available

```
Working detected -> 1
Sangram status: Available
Senthil status: Away
Working detected -> 2
Resetting counter
Senthil status: Away
Resetting counter
sejal status: Available
Working detected -> 3
sejal status: Available
Working detected -> 4
Sangram status: Available
Working detected -> 3
Senthil status: Away
Resetting counter
Sangram status: Available
Working detected -> 4
sejal status: Away
Senthil status: Away
Resetting counter
Resetting counter
sejal status: Away
Resetting counter
Sangram status: Available
Working detected -> 5
Sangram status: Available
Senthil status: Away
Working detected -> 6
Reminder triggered for Sangram
```

```
Resetting counter
Senthil status: Away
Resetting counter
sejal status: Away
Resetting counter
sejal status: Away
Resetting counter
Sangram status: Available
Working detected -> 1
Senthil status: Away
Resetting counter
Sangram status: Available
Working detected -> 2
sejal status: Away
Resetting counter
Senthil status: Away
Resetting counter
sejal status: Away
Resetting counter
Sangram status: Available
Working detected -> 3
Senthil status: Away
Resetting counter
Sangram status: Available
Working detected -> 4
sejal status: Available
Working detected -> 1
Senthil status: Away
Resetting counter
```

sejal status: Available
Working detected -> 2
Sangram status: Available
Working detected -> 5
Senthil status: Away
Sangram status: Available
Resetting counter
Working detected -> 6
Reminder triggered for Sangram
Senthil status: Away
sejal status: Available
Resetting counter
Working detected -> 3
sejal status: Available
Working detected -> 4
Sangram status: Available
Working detected -> 1
Senthil status: Away
Resetting counter
Sangram status: Available
Working detected -> 2
Senthil status: Away
sejal status: Away
Resetting counter
Resetting counter
sejal status: Away
Resetting counter
"

this is my backend terminal where which presense fetching logs are

visible
and when i put any message on wellnessreminderbot i see below response in terminal
"
------ Teams Message Received ------
User Name: Sangram Bhosale
User ID: 29:1QIyJZqWjHeDc_cj9iUvwYAG42zOPUIvBtiGI4d1KchVsOBzqJAPddTF_TVAMSEMH3cq_4HjnIA0Ci7CrX2tE-A
Conversation ID: a:1ODVxDwJUuzE_tA_hIZ_rH7dF3VrK3JWKE8SBbhVhwh_xgBPczYi6i8_ofNuTAF5aDn71OsGoqhclAtvKCEVZ5n7PTA9yGSQ1psECnp5ATMGrPWOr-tpldVUL5GewYpUU
Tenant ID: 1d260f15-bd09-4056-ab2c-5e73843d7099
Message: hi

Reminder triggered for sejal
Internal Server Error: /api/bot/messages/
Traceback (most recent call last):
  File "D:\NOTIFY\environment\Lib\site-packages\django\core\handlers\exception.py", line 55, in inner
    response = get_response(request)
               ^^^^^^^^^^^^^^^^^^^^^^^^
  File "D:\NOTIFY\environment\Lib\site-packages\django\core\handlers\base.py", line 197, in _get_response
    response = wrapped_callback(request, *callback_args, **callback_kwargs)

```
^^^
```

```
    return await self.process_activity_with_identity(activity, identity, logic)

^^^
```

```
  File "D:\NOTIFY\wellnessreminder\core\views.py", line 91, in
call_bot_logic
    await turn_context.send_activity(reply)
  File "D:\NOTIFY\environment\Lib\site-
packages\botbuilder\core\turn_context.py", line 174, in send_activity
    result = await self.send_activities([activity_or_text])

^^^
```

```
packages\botbuilder\core\bot_framework_adapter.py", line 713, in
send_activities
    response = await client.conversations.reply_to_activity(

^^^
```

```
  File "D:\NOTIFY\environment\Lib\site-
packages\django\core\handlers\exception.py", line 55, in inner
    response = get_response(request)
               ^^^^^^^^^^^^^^^^^^^^^^^
  File "D:\NOTIFY\environment\Lib\site-
packages\django\core\handlers\base.py", line 197, in _get_response
    response = wrapped_callback(request, *callback_args,
**callback_kwargs)

^^^^^^^^^^^^^^^^^^^
```

```
packages\django\views\decorators\csrf.py", line 60, in _view_wrapper
    return await view_func(request, *args, **kwargs)
           ^^^
```

```
    return await self.receive_activity_internal(context, callback)

^^^
```

```
    responses = await self.adapter.send_activities(self, output)

^^^
```

Yes — I fully understand what you've built, and I can already see **exactly why it is failing**.
You are actually **very close**. Your presence polling is working. The **main blocker is the Teams bot authentication / app registration mismatch**, and then after that you need **conversation reference storage** for proactive reminder messages.

✅ What is working already

From your logs and code:

- Django backend is running
- ngrok is exposing your local endpoint
- Teams is successfully sending incoming activities to:
  - `POST /api/bot/messages/`
- Your bot **receives** the message from Teams correctly:
  - user name
  - conversation id
  - tenant id
  - message text
- Microsoft Graph presence API is working
- Your scheduler is working
- Reminder trigger logic is working

So the pipeline is currently:

Teams → Azure Bot Service → ngrok → Django → your bot logic ✅

The only place failing is:

Django bot → Azure Bot connector → Teams reply back to user ❌

And your exact error confirms that:

```
ErrorResponseException: Operation returned an invalid status code 'Unauthorized'
```

That means:

- Teams can **call your bot**
- but your bot **cannot authenticate correctly when sending the reply back through Bot Framework**

🎯 Root cause (most important)

There are **TWO separate identity systems** in your project:

1) Microsoft Graph App Registration

Used for:

- `GRAPH_CLIENT_ID`
- `GRAPH_CLIENT_SECRET`
- `GRAPH_TENANT_ID`

This is used only for:

- calling Graph API
- fetching presence

2) Bot / Microsoft App ID credentials

Used for:

- `BOT_APP_ID`
- `BOT_APP_PASSWORD`

This is used only for:

- Bot Framework authentication
- replying to Teams messages
- proactive messaging

👉 **These MUST be the credentials of the Azure Bot's Microsoft App registration**, **not** the Graph app credentials (unless you intentionally used the same app registration for both and configured it correctly).

🔥 What your error means exactly

This line is the smoking gun:

Python Run

and then:

```
botbuilder.schema._models_py3.ErrorResponseException: Operation returned an invalid status
```

This means:

- `process_activity()` accepted the incoming request (so inbound validation is fine enough)
- but when SDK tried to call:

  - `reply_to_activity(...)`
- the connector rejected the bot's access token

That usually happens because one of these is wrong:

1. `BOT_APP_ID` is wrong
2. `BOT_APP_PASSWORD` is wrong
3. Bot endpoint in Azure points to one app, but your code uses another app ID
4. Secret is expired / regenerated
5. Wrong bot registration type
6. App ID in Teams app manifest doesn't match the Azure Bot app ID
7. Tenant policy / custom app policy blocked the bot ("tenant admin disabled this bot")

🧠 You actually have 2 separate issues

Based on your screenshots + logs, you have:

Issue 1: Teams channel admin policy issue

Your Azure screenshot shows:

- **Microsoft Teams → Issues (1)**
- Message:
  - **"The tenant admin disabled this bot"**

This is a **tenant policy / Teams app governance** problem.

Microsoft docs say Teams apps can be blocked by:

- Org-wide app settings
- App permission policies (older model)
- App-centric management (newer model)
- Custom app upload restrictions
- App being blocked in Manage apps
- App not allowed for your user

This is documented in Microsoft Learn and Q&A discussions. The error is literally tenant-level blocking or app-level block in Teams admin governance. Microsoft Learn +3

Issue 2: Bot authentication mismatch

Even after Teams reaches your endpoint, your reply fails with `Unauthorized`.

This is **independent of presence logic**.
This is specifically a **Bot Framework credential / app registration mismatch** issue.

🚨 First correction: your scheduler log is misleading

In `scheduler.py`:

```
scheduler.add_job(
    check_employee_presence,
    'interval',
    minutes=2,
    id='presence_check_job',
```

```
    replace_existing=True
)
logger.info("Presence scheduler started (every 10 minutes)")
```

You configured:

- `minutes=2`

But log says:

- `every 10 minutes`

Fix:

Python Run

This won't break anything, but it can confuse debugging badly.

🚨 Second correction: your "1 hour" logic currently equals 12 minutes, not 1 hour

You said:

> if employee is working straight for 1 hour trigger reminder

But your current logic:

- scheduler runs every **2 minutes**
- reminder triggers at:

Python Run

- 6 × 2 minutes = **12 minutes**

If you want 1 hour:

- 60 / 2 = **30 checks**

So change:

Python Run

✅ Step-by-step: how to make reply to chatbot work

Now I'll give you the exact order to fix this.

STEP 1 — Verify Bot credentials in `.env`

Your `teams_bot.py`:

```python
BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
```

Python Run

These **must be from the Azure Bot's Microsoft App registration**.

In Azure Portal, verify:

Go to your Azure Bot resource:

- **wellnessreminder-bot**
- **Configuration**

Check:

- **Microsoft App ID** ← this must equal `BOT_APP_ID`

Then go to the associated **App Registration** in Entra ID:

- **Certificates & secrets**
- create a **new client secret** if unsure
- copy **secret VALUE** (not secret ID)

Then set:

```
BOT_APP_ID=<Azure Bot Microsoft App ID>
BOT_APP_PASSWORD=<Client Secret VALUE of that same app registration>

GRAPH_CLIENT_ID=<Graph app id or same app if intentionally reused>
GRAPH_CLIENT_SECRET=<Graph secret>
GRAPH_TENANT_ID=<tenant id>
```

⚠️ Very common mistake

People often do one of these:

- put **Secret ID** instead of **Secret Value**
- put Graph app credentials into bot credentials
- use old/expired secret
- use bot resource app id but secret from another app registration

That will produce exactly your error.

STEP 2 — Confirm Teams app manifest uses the same bot ID

Your Teams app package (`manifest.json`) must contain the **same bot ID**.

In `manifest.json`, check:

```
"bots": [
  {
    "botId": "YOUR_BOT_APP_ID",
    "scopes": ["personal"]
  }
]
```

and usually:

```
"webApplicationInfo": {
  "id": "YOUR_BOT_APP_ID",
  "resource": "api://botid-YOUR_BOT_APP_ID"
```

JSON

(Depending on how you packaged the app)

Rule:

- `manifest botId`
- Azure Bot **Microsoft App ID**
- `.env BOT_APP_ID`

➡️ all must match exactly

If they do not match, Teams may send but reply/auth breaks or app behaves inconsistently.

STEP 3 — Fix Teams Admin Center policy issue ("tenant admin disabled this bot")

This is very likely why Azure shows:

> Microsoft Teams → Issues (1)

You (or your Teams admin) must check **all of these**:

A. Teams Admin Center → Teams apps → Manage apps

Find your custom app / bot app.

Check if status is:

- **Allowed** ✅

  not:

- Blocked ❌

Microsoft Learn: app can be explicitly allowed/blocked here. Microsoft Learn +1

B. Teams Admin Center → Teams apps → Permission policies

If your tenant still uses older policy model:

- Open **Global (Org-wide default)** or assigned custom policy
- For **Custom apps**:
  - choose:
    - **Allow all apps**
    - or **Allow specific apps and block all others** and include your app

Microsoft documents this clearly. Microsoft Learn

C. Teams Admin Center → Teams apps → Setup policies

For the user testing the bot (you):

- Ensure **Upload custom apps** = **On**

Docs note that **bots in custom apps only work if custom app upload is allowed**. Microsoft Learn +1

D. Teams Admin Center → Teams apps → Org-wide app settings

Check:

- **Allow interaction with custom apps** = On

- Custom apps not globally disabled

This is important for sideloaded/internal apps. Microsoft's custom app governance docs mention this.

Microsoft Learn

E. If tenant uses new "App centric management"

Microsoft says **App permission policies are deprecated** in some tenants, and app access may be controlled via **app assignment / app-centric management** instead. Microsoft Learn

So if you don't see old behavior working:

- In **Manage apps**
- open the app details
- explicitly **Allow**
- assign to users if your org requires it

STEP 4 — Remove risky permission from the app registration if present

There's a known case reported by developers where:

- app had `Teamwork.Migrate.All`
- bot got blocked in Teams with:
  - **"The tenant admin disabled this bot"**

Microsoft Q&A reports removing `Teamwork.Migrate.All` fixed it for some cases. Microsoft Learn

So check your Entra app registration → API permissions

If you see:

- `Teamwork.Migrate.All`

👉 remove it unless you truly need migration APIs.

For your bot, you probably do **NOT** need it.

STEP 5 — Update Azure Bot messaging endpoint every time ngrok URL changes

Since you are using ngrok, each new tunnel changes URL.

In Azure Bot resource:

- **Settings / Configuration**
- **Messaging endpoint**

Set exactly:

```
https://<your-ngrok-url>/api/bot/messages
```

Then save.

If this is stale, incoming messages may behave inconsistently.

STEP 6 — Improve your `bot_messages` endpoint to avoid 500 crash

Right now if send fails, Django returns 500 and Azure logs show “BadRequest” / issues.

You should **catch bot framework exceptions** and return a clean JSON response.

Updated `views.py` bot handler (safe version)

```
@csrf_exempt
async def bot_messages(request):
    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"}, status=405)

    try:
        body = json.loads(request.body)
        activity = Activity().deserialize(body)
        auth_header = request.headers.get("Authorization", "")

        async def call_bot_logic(turn_context: TurnContext):
            user_text = turn_context.activity.text or ""
            user_id = turn_context.activity.from_property.id
            user_name = turn_context.activity.from_property.name
            conversation_id = turn_context.activity.conversation.id
            tenant_id = getattr(turn_context.activity.conversation, "tenant_id", None)
            service_url = turn_context.activity.service_url

            print("\n------ Teams Message Received ------")
            print("User Name:", user_name)
            print("User ID:", user_id)
            print("Conversation ID:", conversation_id)
            print("Tenant ID:", tenant_id)
            print("Service URL:", service_url)
            print("Message:", user_text)
            print("------------------------------------\n")
```

```
            # TODO: store conversation reference here (important for reminders later)

            if user_text:
                reply = f"Hello {user_name} 👋 I received: {user_text}"
                try:
                    await turn_context.send_activity(reply)
                except Exception as send_error:
                    print("BOT SEND ERROR:", str(send_error))
                    # don't crash whole request
                    # optionally re-raise during debugging
                    raise

        await ADAPTER.process_activity(activity, auth_header, call_bot_logic)
        return JsonResponse({"status": "ok"})

    except Exception as e:
        print("BOT MESSAGE ERROR:", str(e))
        return JsonResponse({"error": str(e)}, status=500)
```

This won't solve auth, but it makes debugging much cleaner.

🚨 MOST IMPORTANT ARCHITECTURE GAP FOR REMINDERS

Even after chatbot reply starts working, **your reminder logic still cannot proactively send to employees yet**.

Why?

Because Teams proactive messages require:

- `conversation_id`
- `tenant_id`

- `service_url`
- preferably full **conversation reference**

Microsoft docs are explicit:

- You **must store conversation reference / conversationId**
- You cannot send proactive messages just using email or UPN
- For proactive Teams messages, you need `conversationId`, `tenantId`, and `serviceUrl` (or create conversation flow) Microsoft Learn +2

🧱 What you need to add to your database

Your `Employee` model currently has:

```python
azure_user_id
display_name
```

But for proactive bot messages you also need to store:

- Teams user ID (`29:...`)
- conversation ID
- tenant ID
- service URL

Add these fields to `Employee`

```
class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    azure_user_id = models.CharField(max_length=255, unique=True)
    display_name = models.CharField(max_length=255)
    email = models.EmailField()

    teams_user_id = models.CharField(max_length=500, null=True, blank=True)
    teams_conversation_id = models.TextField(null=True, blank=True)
    teams_tenant_id = models.CharField(max_length=255, null=True, blank=True)
    teams_service_url = models.TextField(null=True, blank=True)

    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)
    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name
```

Then run:

```bash
python manage.py makemigrations
```

STEP 7 — Save conversation info when user messages the bot

When user sends “hi”, store the Teams conversation details.

In `call_bot_logic()`:

Python Run

Then:

```python
aad_object_id = getattr(turn_context.activity.from_property, "aad_object_id", None)
teams_user_id = turn_context.activity.from_property.id
conversation_id = turn_context.activity.conversation.id
tenant_id = getattr(turn_context.activity.conversation, "tenant_id", None)
service_url = turn_context.activity.service_url
user_name = turn_context.activity.from_property.name

employee = None

if aad_object_id:
    employee = Employee.objects.filter(azure_user_id=aad_object_id).first()

if employee:
    employee.teams_user_id = teams_user_id
    employee.teams_conversation_id = conversation_id
    employee.teams_tenant_id = tenant_id
    employee.teams_service_url = service_url
    employee.save(update_fields=[
        "teams_user_id",
        "teams_conversation_id",
        "teams_tenant_id",
        "teams_service_url"
    ])
    print(f"Stored Teams conversation info for {employee.display_name}")
```

```
else:
    print(f"No employee found for AAD object ID: {aad_object_id}")
```

❗ IMPORTANT: use `aad_object_id`, not Teams `29:...` ID as employee mapping

You currently store:

- `azure_user_id`

This should ideally be the **Entra AAD object ID** of the user.

When Teams sends activity, `from_property.id` is often:

- `29:xxxxx` → Teams-specific ID

But for stable identity mapping, better use:

- `turn_context.activity.from_property.aad_object_id`

That often matches the Entra user object id (what you should store in `Employee.azure_user_id`).

So when mapping bot user ↔ employee, use:

Python Run

```
```

This is much better than `29:...`

STEP 8 — Proactive reminder sending (after bot auth is fixed)

Once replies work, then implement proactive messaging.

You need to call the bot adapter using stored conversation reference.

A simple pattern in Python Bot Framework is:

- create `ConversationReference`
- call `continue_conversation`

Example proactive send helper

Create `core/services/teams_sender.py`

```
from botbuilder.schema import ConversationReference, ChannelAccount, ConversationAccount
from botbuilder.core import TurnContext
from core.services.teams_bot import ADAPTER, BOT_APP_ID


async def send_proactive_message(employee, text):
    if not all([
        employee.teams_user_id,
        employee.teams_conversation_id,
        employee.teams_service_url
    ]):
        raise Exception(f"Missing Teams conversation info for {employee.display_name}")

    conversation_reference = ConversationReference(
        channel_id="msteams",
```

```
        service_url=employee.teams_service_url,
        bot=ChannelAccount(id=BOT_APP_ID),
        user=ChannelAccount(id=employee.teams_user_id),
        conversation=ConversationAccount(
            id=employee.teams_conversation_id,
            tenant_id=employee.teams_tenant_id,
            is_group=False
        )
    )

    async def callback(turn_context: TurnContext):
        await turn_context.send_activity(text)

    await ADAPTER.continue_conversation(
        conversation_reference,
        callback,
        BOT_APP_ID
    )
```

⚠️ But your scheduler is synchronous

Your `check_employee_presence()` is sync, while bot send is async.

So when reminder triggers, do:

```python
import asyncio
```

Python Run

Inside reminder block:

```
if tracker.consecutive_available_count >= 30:
    message = "⏰ Time for a quick wellness break! Drink water and stretch yourself."

    ReminderLog.objects.create(
        employee=employee,
        message=message
    )

    print(f"Reminder triggered for {employee.display_name}")

    try:
        asyncio.run(send_proactive_message(employee, message))
        print(f"Reminder sent to Teams for {employee.display_name}")
    except Exception as e:
        print(f"Failed to send Teams reminder to {employee.display_name}: {e}")

    tracker.consecutive_available_count = 0
```

⚠️ Better design suggestion (important)

Since you already have **Celery in settings**, long-term I strongly recommend:

- use **Celery Beat** for schedule
- use **Celery task** for presence polling
- use **async_to_sync / event loop safe wrapper** for bot send
- avoid APScheduler inside Django dev server in production

Because:

- APScheduler inside Django can start multiple times

- autoreload can duplicate jobs
- dev server can cause weird behavior

Right now you have **both**:

- APScheduler
- Celery Beat config

That's not ideal.

Pick one:

✅ Best for Django production: **Celery Beat**

🔴 One more important code issue: duplicate scheduler runs

Your logs show weird interleaving like:

- Sangram status
- Senthil status
- then "Resetting counter"
- then another user

This can happen because:

- APScheduler job overlap
- Django dev autoreload starts multiple schedulers
- scheduler starts twice from app startup

If your scheduler is initialized in `AppConfig.ready()` (not shown, but likely), Django dev server can start it twice.

Protect against this in development

If you start scheduler from `apps.py`, do something like:

```
import os
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        if os.environ.get("RUN_MAIN") == "true":
            from core.scheduler import start
            start()
```

This avoids duplicate startup due to Django autoreloader.

🧪 Exact debugging checklist for you (in order)

Do these in this exact order:

---

1. Recreate bot secret

In Entra app registration for the bot:

- New client secret

- copy **VALUE**
- update `.env`

---

2. Confirm `.env`

Print at startup:

```python
print("BOT APP ID:", BOT_APP_ID)
print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else "NO")
```

Python Run

You already do this — good.

---

3. Ensure these 3 IDs match exactly

- Azure Bot → Microsoft App ID
- `.env BOT_APP_ID`
- Teams app manifest → `botId`

If any mismatch → fix and re-upload app package.

---

4. Update Azure Bot messaging endpoint

Every time ngrok changes:

```
https://<ngrok-domain>/api/bot/messages
```

---

5. In Teams Admin Center

Check:

- Manage apps → your app → **Allowed**
- Setup policies → **Upload custom apps = On**
- Permission policies / app-centric assignment → your app allowed
- Org-wide app settings → custom apps allowed

---

6. Remove risky API permission

If app registration has:

- `Teamwork.Migrate.All`

remove it.

---

7. Clear Teams cache / reinstall app

Sometimes Teams caches bad manifest/app state.

Do:

- uninstall bot from Teams personal apps
- delete old uploaded custom app package

- upload fresh package again
- sign out/in if needed

---

8. Test Web Chat in Azure

If Azure "Test in Web Chat" can send and receive:

- Bot auth is usually okay
- then Teams policy/manifest issue remains

If Web Chat also fails:

- bot credentials definitely wrong

💡 My diagnosis of your current situation (most likely)

Based on everything you shared, I believe this is the most likely order of problems:

Primary blocker:

`BOT_APP_ID` / `BOT_APP_PASSWORD` **do not match the Azure Bot's actual app registration credentials**
➡️ causing `Unauthorized` on reply

Secondary blocker:

Teams admin policy / app governance is blocking or partially blocking the custom bot app
➡️ Azure Teams channel shows:

- "The tenant admin disabled this bot"

After those are fixed:

You still need:

- store `aad_object_id`
- store `conversation_id`
- store `tenant_id`
- store `service_url`
- implement proactive `continue_conversation`

✅ What I recommend you do next (best next move)

If you want, I can now give you the **FULLY UPDATED WORKING VERSION** of these files for your project:

1. `models.py`
   - add Teams conversation fields
2. `views.py`
   - fix bot reply handling
   - store Teams conversation info from incoming message
3. `teams_bot.py`
   - robust adapter with error handling
4. `presence_engine.py`
   - 1-hour correct logic
   - trigger proactive reminder send
5. `teams_sender.py` (new file)
   - send proactive Teams message to stored conversation
6. `apps.py`

    - safe scheduler start without duplicate runs
7. `scheduler.py`
    - clean version

🚀 My strong suggestion

Reply with exactly this:

“give me full updated files now”

and I will give you:

- **production-style corrected code**
- file by file
- ready to paste
- with:
    - chatbot reply working structure
    - proactive reminder architecture
    - proper employee mapping using `aad_object_id`
    - 1-hour reminder logic
    - safe scheduler start

That will be the fastest way to get your bot working properly.

Sources

okay i have read most of the above things now i want you to tell me what programatical changes to do step by step in detail proparlly with path then we proceed next

Perfect.
Now we'll do this **the right way, step by step, with exact file paths and exact code changes — only programmatic changes first**.

👉 **Important:** These code changes will prepare everything correctly, but **your bot reply will still fail until Azure/Teams configuration is fixed** (BOT credentials + Teams admin allow).
Still, we should make the code **100% correct first** so once Azure side is fixed, it works immediately.

✅ Goal of this phase

We will make these programmatic changes:

1. **Store Teams conversation details** when a user messages the bot
2. **Use AAD Object ID** (stable identity) to map Teams user ↔ Employee
3. **Add proactive messaging support** (for reminders)
4. **Fix reminder logic** for real 1 hour
5. **Make bot handler safer** (no ugly 500 crash without logs)
6. **Keep scheduler clean and accurate**

🧭 IMPORTANT: What we will change now

We will update these files:

- `core/models.py`
- `core/serializers.py`
- `core/views.py`
- `core/services/teams_bot.py`
- `core/services/presence_engine.py`
- `core/scheduler.py`

We will create **new file**:

- `core/services/teams_sender.py`

And later we'll do migrations.

STEP 1 — Update `core/models.py`

📍 File path:

`core/models.py`

✅ Why this change?

Right now you only store:

- `azure_user_id`
- `display_name`
- `email`

But to send **proactive Teams reminders**, you must also store:

- `teams_user_id`
- `teams_conversation_id`
- `teams_tenant_id`
- `teams_service_url`

Without these, reminder cannot be sent back to Teams.

🔁 Replace your `core/models.py` with this:

```
import uuid
from django.db import models


class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

    # This should store the Entra / Azure AD Object ID of the user
    azure_user_id = models.CharField(max_length=255, unique=True)

    display_name = models.CharField(max_length=255)
    email = models.EmailField()

    # Teams conversation details (captured when user sends first message to bot)
    teams_user_id = models.CharField(max_length=500, null=True, blank=True)
    teams_conversation_id = models.TextField(null=True, blank=True)
    teams_tenant_id = models.CharField(max_length=255, null=True, blank=True)
    teams_service_url = models.TextField(null=True, blank=True)
```

```
    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)

    # Reminder interval in minutes (example: 60 for 1 hour)
    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name


class PresenceLog(models.Model):
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE)
    status = models.CharField(max_length=50)
    checked_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.employee.display_name} - {self.status} @ {self.checked_at}"


class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee, on_delete=models.CASCADE)
    consecutive_available_count = models.IntegerField(default=0)
    last_status = models.CharField(max_length=50, null=True, blank=True)
    reminder_sent = models.BooleanField(default=False)

    def __str__(self):
        return f"{self.employee.display_name} tracker"


class ReminderLog(models.Model):
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE)
    sent_at = models.DateTimeField(auto_now_add=True)
    message = models.TextField()
```

```
    def __str__(self):
        return f"Reminder to {self.employee.display_name} @ {self.sent_at}"
```

STEP 2 — Update `core/serializers.py`

📍 File path:

`core/serializers.py`

✅ Why?

So the new Teams fields are visible in API if needed.

🔁 Replace your `core/serializers.py` with this:

```
from rest_framework import serializers
from core.models import Employee, PresenceLog, ReminderLog


class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = "__all__"


class PresenceLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = PresenceLog
```

```
        fields = "__all__"


class ReminderLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = ReminderLog
        fields = "__all__"
```

👉 No major change needed here, `fields="__all__"` already covers it — but keep this clean version.

STEP 3 — Create proactive sender service

📍 New file path:

`core/services/teams_sender.py`

✅ Why?

This file will be used when reminder triggers.

It will send a message to a specific user **without them typing at that moment** (proactive message).

This is required for:

- "Drink water"
- "Stretch yourself"
- "Take a break"

➕ Create new file `core/services/teams_sender.py`

```python
from botbuilder.schema import (
    ConversationReference,
    ChannelAccount,
    ConversationAccount,
)
from botbuilder.core import TurnContext

from core.services.teams_bot import ADAPTER, BOT_APP_ID


async def send_proactive_message(employee, text: str):
    """
    Send a proactive Teams message to an employee using stored conversation details.
    """

    if not employee.teams_user_id:
        raise Exception(f"Missing teams_user_id for {employee.display_name}")

    if not employee.teams_conversation_id:
        raise Exception(f"Missing teams_conversation_id for {employee.display_name}")

    if not employee.teams_service_url:
        raise Exception(f"Missing teams_service_url for {employee.display_name}")

    conversation_reference = ConversationReference(
        channel_id="msteams",
        service_url=employee.teams_service_url,
        bot=ChannelAccount(id=BOT_APP_ID),
        user=ChannelAccount(id=employee.teams_user_id),
        conversation=ConversationAccount(
            id=employee.teams_conversation_id,
            tenant_id=employee.teams_tenant_id,
            is_group=False
```

```
        ),
    )

    async def callback(turn_context: TurnContext):
        await turn_context.send_activity(text)

    await ADAPTER.continue_conversation(
        conversation_reference,
        callback,
        BOT_APP_ID,
    )
```

STEP 4 — Improve bot adapter file

📍 File path:

`core/services/teams_bot.py`

✅ Why?

Your current file is mostly okay, but we want:

- clean env loading
- proper startup logs
- reusable `BOT_APP_ID`
- better debugging

🔁 Replace `core/services/teams_bot.py` with this:

```python
import os
from dotenv import load_dotenv
from botbuilder.core import BotFrameworkAdapterSettings, BotFrameworkAdapter

load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()

print("BOT APP ID:", BOT_APP_ID)
print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else "NO")
print("BOT PASSWORD LENGTH:", len(BOT_APP_PASSWORD) if BOT_APP_PASSWORD else 0)

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD
)

ADAPTER = BotFrameworkAdapter(SETTINGS)
```

STEP 5 — Update bot message handler to store Teams conversation info

📍 File path:

`core/views.py`

✅ Why this is critical?

When user sends `"hi"` to bot, Teams gives you:

- `from_property.id` → Teams user ID (`29:...`)
- `from_property.aad_object_id` → AAD / Entra Object ID (**best for mapping to Employee**)
- `conversation.id`
- `conversation.tenant_id`
- `service_url`

You must store these for later reminders.

🔁 Replace your `core/views.py` with this full version

```
import json

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

from botbuilder.schema import Activity
from botbuilder.core import TurnContext

from core.models import Employee
from core.serializers import EmployeeSerializer
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence
from core.services.teams_bot import ADAPTER
```

```python
class EmployeeListCreate(APIView):

    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)


class TestPresence(APIView):

    def get(self, request):
        employee = Employee.objects.first()

        if not employee:
            return Response({"error": "No employee found"}, status=404)

        data = get_user_presence(employee.azure_user_id)
        return Response(data)


class RunPresenceCheck(APIView):

    def post(self, request):
        check_employee_presence()
```

```
        return Response({"message": "Presence check executed"})


@csrf_exempt
async def bot_messages(request):
    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"}, status=405)

    try:
        body = json.loads(request.body)
        activity = Activity().deserialize(body)
        auth_header = request.headers.get("Authorization", "")

        async def call_bot_logic(turn_context: TurnContext):
            user_text = (turn_context.activity.text or "").strip()

            user_name = turn_context.activity.from_property.name
            teams_user_id = turn_context.activity.from_property.id

            # VERY IMPORTANT: this is the stable Azure AD / Entra object ID
            aad_object_id = getattr(turn_context.activity.from_property, "aad_object_id", None)

            conversation_id = turn_context.activity.conversation.id
            tenant_id = getattr(turn_context.activity.conversation, "tenant_id", None)
            service_url = turn_context.activity.service_url

            print("\n------ Teams Message Received ------")
            print("User Name:", user_name)
            print("Teams User ID:", teams_user_id)
            print("AAD Object ID:", aad_object_id)
            print("Conversation ID:", conversation_id)
            print("Tenant ID:", tenant_id)
            print("Service URL:", service_url)
            print("Message:", user_text)
            print("------------------------------------\n")
```

```
            # Match employee by Azure AD object ID
            employee = None
            if aad_object_id:
                employee = Employee.objects.filter(azure_user_id=aad_object_id).first()

            if employee:
                employee.teams_user_id = teams_user_id
                employee.teams_conversation_id = conversation_id
                employee.teams_tenant_id = tenant_id
                employee.teams_service_url = service_url
                employee.save(update_fields=[
                    "teams_user_id",
                    "teams_conversation_id",
                    "teams_tenant_id",
                    "teams_service_url",
                ])
                print(f"Stored Teams conversation details for {employee.display_name}")
            else:
                print(f"No employee matched with AAD Object ID: {aad_object_id}")

            if user_text:
                reply = f"Hello {user_name} 👋 I received: {user_text}"

                try:
                    await turn_context.send_activity(reply)
                    print("Reply sent successfully")
                except Exception as send_error:
                    print("BOT SEND ERROR:", str(send_error))
                    raise

        await ADAPTER.process_activity(
            activity,
            auth_header,
            call_bot_logic
```

```
        )

        return JsonResponse({"status": "ok"})

    except Exception as e:
        print("BOT MESSAGE ERROR:", str(e))
        return JsonResponse({"error": str(e)}, status=500)
```

STEP 6 — Update presence engine for real 1-hour logic + proactive reminder

📍 File path:

`core/services/presence_engine.py`

✅ Why?

Right now:

- scheduler every 2 min
- count >= 6

That means **12 minutes**, not 1 hour.

We will make it:

- every 2 min
- dynamic threshold based on `reminder_interval_minutes`
- example: 60 min → 30 checks

We will also:

- send proactive Teams reminder when threshold hits
- safely handle async send

🔁 Replace `core/services/presence_engine.py` with this:

```
import asyncio

from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting"
]

# This must match scheduler interval
CHECK_INTERVAL_MINUTES = 2


def check_employee_presence():
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )
```

```
    for employee in employees:
        try:
            presence_data = get_user_presence(employee.azure_user_id)
            status = presence_data.get("availability")

            print(f"{employee.display_name} status: {status}")

        except Exception as e:
            print(f"Presence fetch failed for {employee.display_name}: {e}")
            continue

        # Save presence log
        PresenceLog.objects.create(
            employee=employee,
            status=status
        )

        tracker, _ = ActivityTracker.objects.get_or_create(
            employee=employee
        )

        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected -> {tracker.consecutive_available_count}")

        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Calculate required checks dynamically based on employee's reminder interval
        # Example: 60 min / 2 min = 30 checks
        required_checks = max(1, employee.reminder_interval_minutes // CHECK_INTERVAL_MINUTES)

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = "⏰ Time for a quick wellness break! Drink water and stretch yours
```

```
            ReminderLog.objects.create(
                employee=employee,
                message=reminder_message
            )

            print(f"Reminder triggered for {employee.display_name}")

            # Try sending proactive Teams message
            try:
                asyncio.run(send_proactive_message(employee, reminder_message))
                print(f"Reminder sent to Teams for {employee.display_name}")
            except Exception as e:
                print(f"Failed to send Teams reminder to {employee.display_name}: {e}")

            # Reset after reminder
            tracker.consecutive_available_count = 0

        tracker.last_status = status
        tracker.save()
```

STEP 7 — Clean scheduler file

📍 File path:

`core/scheduler.py`

✅ Why?

Your scheduler says every 2 minutes but logs “10 minutes”.

We'll make it accurate and clean.

🔁 Replace `core/scheduler.py` with this:

```
from apscheduler.schedulers.background import BackgroundScheduler
from core.services.presence_engine import check_employee_presence
import logging

logger = logging.getLogger(__name__)

scheduler = BackgroundScheduler()


def start():
    if not scheduler.running:
        scheduler.add_job(
            check_employee_presence,
            'interval',
            minutes=2,
            id='presence_check_job',
            replace_existing=True
        )
        scheduler.start()
        logger.info("Presence scheduler started (every 2 minutes)")
```

STEP 8 — (Optional but strongly recommended) Safe scheduler startup in `core/apps.py`

You did not share `core/apps.py`, but this is **very important**.

If scheduler starts inside Django app startup, Django dev server can start it twice because of auto-reload.

📍 File path:

`core/apps.py`

🔁 Replace or update with this:

```
import os
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        # Prevent duplicate scheduler start in Django dev autoreload
        if os.environ.get("RUN_MAIN") == "true":
            from core.scheduler import start
            start()
```

STEP 9 — Run migrations

Since we added new fields in `Employee`, you **must** migrate.

Run in terminal:

```bash
python manage.py makemigrations
```

STEP 10 — Verify your `Employee.azure_user_id` values are correct

🚨 Super important

Your bot logic now matches employees using:

```python
```

That means in your `Employee` table:

`azure_user_id` should store:

✅ **Azure AD / Entra Object ID of the user**

NOT:

- email
- Teams `29:...` id
- UPN
- display name

How to verify?

When you message the bot after this code change, terminal will print:

```
AAD Object ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Take that value and compare with `Employee.azure_user_id`.

If they don't match, no employee will be linked, and reminders won't have conversation details stored.

STEP 11 — What will happen after these code changes?

After all above code is in place:

When user messages bot:

- bot receives message
- prints Teams metadata
- matches employee by `aad_object_id`
- stores:
  - `teams_user_id`
  - `teams_conversation_id`
  - `teams_tenant_id`
  - `teams_service_url`
- tries to reply

If Azure bot auth is fixed:

- user gets reply immediately

Later when reminder triggers:

- presence engine checks status
- if worked for enough time
- creates `ReminderLog`
- uses stored Teams conversation details
- sends proactive reminder

⚠️ Important note: reply may still fail right now

Even after all code changes, **this line may still fail** until Azure side is corrected:

```python
```

Why?

Because your current issue is still likely:

- wrong `BOT_APP_ID` / `BOT_APP_PASSWORD`
- or Teams admin disabled bot

But after code is correct, once Azure is fixed, it will work instantly.

✅ Recommended order for you RIGHT NOW

Please do this **exactly in this order**:

1. Update `core/models.py`
2. Update `core/serializers.py`
3. Create `core/services/teams_sender.py`
4. Update `core/services/teams_bot.py`
5. Update `core/views.py`
6. Update `core/services/presence_engine.py`
7. Update `core/scheduler.py`
8. Update `core/apps.py` (if needed)
9. Run migrations
10. Restart Django server
11. Message bot once in Teams
12. Check terminal logs for:

- `AAD Object ID`
- `Stored Teams conversation details for ...`

🔍 What to look for after code update

When you send "hi" to bot, terminal should show something like:

```
------ Teams Message Received ------
User Name: Sangram Bhosale
Teams User ID: 29:xxxxx
AAD Object ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Conversation ID: a:xxxxx
Tenant ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

```
Service URL: https://smba.trafficmanager.net/...
Message: hi
------------------------------------

Stored Teams conversation details for Sangram
BOT SEND ERROR: Operation returned an invalid status code 'Unauthorized'
```

If you see **Stored Teams conversation details**, that means code-side mapping is working perfectly.

Then only Azure side remains.

🚀 Best next step after you do these code changes

After you finish these code changes, send me **only these 3 things**:

1. your updated `core/apps.py`
2. terminal output after sending `"hi"` to bot
3. one screenshot / text of your Azure Bot **Configuration** page values (just names/IDs, hide secrets)

Then I will tell you **the exact Azure-side fixes next**:

- what to check in Bot resource
- what to check in App Registration
- what to check in Teams app manifest
- what to check in Teams Admin Center

✅ If you want fastest progress

If you want, next message you can simply send:

“i updated code, now tell me exact azure portal checks step by step”

and I’ll guide you with **very precise Azure + Teams configuration steps** (no guessing).

after above code update i see this response
System check identified no issues (0 silenced).
March 19, 2026 - 12:26:28
Django version 5.2.11, using settings 'wellnessreminder.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see:
https://docs.djangoproject.com/en/5.2/howto/deployment/

------ Teams Message Received ------
User Name: Sangram Bhosale
Teams User ID:
29:1QIyJZqWjHeDc_cj9iUvwYAG42zOPUIvBtiGI4d1KchVsOBzqJAPddTF_TVAMSEMH3cq_4HjnIA0Ci7CrX2tE-A
AAD Object ID: 16dd0967-ea3e-45b3-b457-22460a66ef2e
Conversation ID:
a:1ODVxDwJUuzE_tA_hIZ_rH7dF3VrK3JWKE8SBbhVhwh_xgBPczYi6i8_ofNuTAF5aDn71OsGoqhclAtvKCEVZ5n7PTA9yGSQ1psECnp5ATMGrPWOr-tpIdVUL5GewYpUU

Tenant ID: 1d260f15-bd09-4056-ab2c-5e73843d7099
Service URL: https://smba.trafficmanager.net/in/1d260f15-bd09-4056-ab2c-5e73843d7099/
Message: hi

BOT MESSAGE ERROR: You cannot call this from an async context - use a thread or sync_to_async.
Internal Server Error: /api/bot/messages/
[19/Mar/2026 12:26:35] "POST /api/bot/messages/ HTTP/1.1" 500 88

------ Teams Message Received ------
User Name: Sangram Bhosale
Teams User ID:
29:1QlyJZqWjHeDc_cj9iUvwYAG42zOPUIvBtiGI4d1KchVsOBzqJAPddTF_TVAMSEMH3cq_4HjnIA0Ci7CrX2tE-A
AAD Object ID: 16dd0967-ea3e-45b3-b457-22460a66ef2e
Conversation ID:
a:1ODVxDwJUuzE_tA_hIZ_rH7dF3VrK3JWKE8SBbhVhwh_xgBPczYi6i8_ofNuTAF5aDn71OsGoqhclAtvKCEVZ5n7PTA9yGSQ1psECnp5ATMGrPWOr-tpldVUL5GewYpUU
Tenant ID: 1d260f15-bd09-4056-ab2c-5e73843d7099
Service URL: https://smba.trafficmanager.net/in/1d260f15-bd09-4056-ab2c-5e73843d7099/
Message: hi

BOT MESSAGE ERROR: You cannot call this from an async context - use a thread or sync_to_async.

Internal Server Error: /api/bot/messages/
[19/Mar/2026 12:26:36] "POST /api/bot/messages/ HTTP/1.1" 500 88

Perfect — this is actually **good progress**.

Now we have hit the **next real code issue**, and this one is **100% Django async/sync problem**, not Azure yet.

✅ What this new error means

You are now getting:

```
BOT MESSAGE ERROR: You cannot call this from an async context - use a thread or sync_to_asyr
```

This means:

- your `bot_messages` view is `async def`
- inside it, you are doing **Django ORM calls directly**
- Django ORM is **sync by default**
- in an async view, direct ORM access throws this exact error

So this part in your `core/views.py` is causing it:

```python
employee = Employee.objects.filter(azure_user_id=aad_object_id).first()
```

Because that is sync ORM inside async context.

🎯 Important conclusion

This means:

- Teams request reaches your bot ✅
- Bot activity parsing works ✅
- Metadata is available ✅
- Your code crashes **before** it even tries to send reply ❌

So currently we have **not yet reached the old Unauthorized reply issue** in this run, because the ORM crash happens first.

✅ What to do now

We need to make your async bot view use Django ORM safely.

Best fix for your current architecture:

Use:

- `asgiref.sync.sync_to_async`

This allows sync DB operations inside async view.

🔧 STEP-BY-STEP FIX (very precise)

We will **only update** `core/views.py` now.

STEP 1 — Update imports in `core/views.py`

📍 File path:

`core/views.py`

Add this import near the top:

Python Run

STEP 2 — Create async-safe helper functions

Inside `core/views.py`, **above** `@csrf_exempt` bot view, add these helper functions.

✅ Add these functions in `core/views.py`

Put them **after** `RunPresenceCheck` class and **before** `@csrf_exempt`.

```python
@sync_to_async
def get_employee_by_aad_object_id(aad_object_id):
    if not aad_object_id:
        return None
    return Employee.objects.filter(azure_user_id=aad_object_id).first()
```

```
@sync_to_async
def store_employee_teams_details(employee, teams_user_id, conversation_id, tenant_id, service_url
    employee.teams_user_id = teams_user_id
    employee.teams_conversation_id = conversation_id
    employee.teams_tenant_id = tenant_id
    employee.teams_service_url = service_url
    employee.save(update_fields=[
        "teams_user_id",
        "teams_conversation_id",
        "teams_tenant_id",
        "teams_service_url",
    ])
```

STEP 3 — Update bot logic to use `await`

Now replace the sync ORM usage inside `call_bot_logic()`.

❌ REMOVE this old block

```
employee = None
if aad_object_id:
    employee = Employee.objects.filter(azure_user_id=aad_object_id).first()

if employee:
    employee.teams_user_id = teams_user_id
    employee.teams_conversation_id = conversation_id
    employee.teams_tenant_id = tenant_id
```

```
    employee.teams_service_url = service_url
    employee.save(update_fields=[
        "teams_user_id",
        "teams_conversation_id",
        "teams_tenant_id",
        "teams_service_url",
    ])
    print(f"Stored Teams conversation details for {employee.display_name}")
else:
    print(f"No employee matched with AAD Object ID: {aad_object_id}")
```

✅ REPLACE with this

```
employee = await get_employee_by_aad_object_id(aad_object_id)

if employee:
    await store_employee_teams_details(
        employee,
        teams_user_id,
        conversation_id,
        tenant_id,
        service_url
    )
    print(f"Stored Teams conversation details for {employee.display_name}")
else:
    print(f"No employee matched with AAD Object ID: {aad_object_id}")
```

STEP 4 — Your fully corrected `core/views.py`

To avoid confusion, I'm giving you the **full corrected file**.

📍 Replace entire `core/views.py` with this:

```
import json

from asgiref.sync import sync_to_async

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

from botbuilder.schema import Activity
from botbuilder.core import TurnContext

from core.models import Employee
from core.serializers import EmployeeSerializer
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence
from core.services.teams_bot import ADAPTER


class EmployeeListCreate(APIView):

    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)
```

```
    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)


class TestPresence(APIView):

    def get(self, request):
        employee = Employee.objects.first()

        if not employee:
            return Response({"error": "No employee found"}, status=404)

        data = get_user_presence(employee.azure_user_id)
        return Response(data)


class RunPresenceCheck(APIView):

    def post(self, request):
        check_employee_presence()
        return Response({"message": "Presence check executed"})


@sync_to_async
def get_employee_by_aad_object_id(aad_object_id):
    if not aad_object_id:
        return None
    return Employee.objects.filter(azure_user_id=aad_object_id).first()
```

```
@sync_to_async
def store_employee_teams_details(employee, teams_user_id, conversation_id, tenant_id, service_url
    employee.teams_user_id = teams_user_id
    employee.teams_conversation_id = conversation_id
    employee.teams_tenant_id = tenant_id
    employee.teams_service_url = service_url
    employee.save(update_fields=[
        "teams_user_id",
        "teams_conversation_id",
        "teams_tenant_id",
        "teams_service_url",
    ])


@csrf_exempt
async def bot_messages(request):
    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"}, status=405)

    try:
        body = json.loads(request.body)
        activity = Activity().deserialize(body)
        auth_header = request.headers.get("Authorization", "")

        async def call_bot_logic(turn_context: TurnContext):
            user_text = (turn_context.activity.text or "").strip()

            user_name = turn_context.activity.from_property.name
            teams_user_id = turn_context.activity.from_property.id

            # Stable Azure AD / Entra object ID
            aad_object_id = getattr(turn_context.activity.from_property, "aad_object_id", None)
```

```
            conversation_id = turn_context.activity.conversation.id
            tenant_id = getattr(turn_context.activity.conversation, "tenant_id", None)
            service_url = turn_context.activity.service_url

            print("\n------ Teams Message Received ------")
            print("User Name:", user_name)
            print("Teams User ID:", teams_user_id)
            print("AAD Object ID:", aad_object_id)
            print("Conversation ID:", conversation_id)
            print("Tenant ID:", tenant_id)
            print("Service URL:", service_url)
            print("Message:", user_text)
            print("------------------------------------\n")

            employee = await get_employee_by_aad_object_id(aad_object_id)

            if employee:
                await store_employee_teams_details(
                    employee,
                    teams_user_id,
                    conversation_id,
                    tenant_id,
                    service_url
                )
                print(f"Stored Teams conversation details for {employee.display_name}")
            else:
                print(f"No employee matched with AAD Object ID: {aad_object_id}")

            if user_text:
                reply = f"Hello {user_name} 👋 I received: {user_text}"

                try:
                    await turn_context.send_activity(reply)
                    print("Reply sent successfully")
                except Exception as send_error:
```

```
                    print("BOT SEND ERROR:", str(send_error))
                    raise

        await ADAPTER.process_activity(
            activity,
            auth_header,
            call_bot_logic
        )

        return JsonResponse({"status": "ok"})

    except Exception as e:
        print("BOT MESSAGE ERROR:", str(e))
        return JsonResponse({"error": str(e)}, status=500)
```

STEP 5 — Restart Django server

After saving file:

```
CTRL + C
```

Bash

STEP 6 — Test again in Teams

Send:

```
hi
```

to the bot.

✅ What you should expect now

If `Employee.azure_user_id` matches the printed AAD Object ID:

You should see:

```
Stored Teams conversation details for Sangram
```

Then after that, **most likely** you will hit the **next old error** again:

```
BOT SEND ERROR: Operation returned an invalid status code 'Unauthorized'
```

That is actually what we want next — because that will confirm:

- async DB issue fixed ✅
- employee mapping fixed ✅
- conversation storage fixed ✅
- now only Azure bot auth remains ❌

🔍 Very important: verify `Employee.azure_user_id`

From your log:

```
AAD Object ID: 16dd0967-ea3e-45b3-b457-22460a66ef2e
```

Your `Employee` record for **Sangram** must have:

```
azure_user_id = 16dd0967-ea3e-45b3-b457-22460a66ef2e
```

If not, it will print:

```
No employee matched with AAD Object ID: ...
```

and reminder sending later won't work.

🚨 One more small note about `presence_engine.py`

Right now in `presence_engine.py` you use:

```python
```

This is okay **for now** because `check_employee_presence()` is sync and runs in scheduler thread.

So no change needed right now.

✅ What to send me next

After you do this, send me the **new terminal output** after messaging the bot once.

I specifically want to see one of these:

Best case:

```
Stored Teams conversation details for Sangram
BOT SEND ERROR: Operation returned an invalid status code 'Unauthorized'
```

or

If employee not matched:

```
No employee matched with AAD Object ID: ...
BOT SEND ERROR: Operation returned an invalid status code 'Unauthorized'
```

🎯 After that, next step

Once you send me that output, I will give you the **exact Azure portal / Teams portal checks** in order:

1. Azure Bot → Configuration
2. App Registration → Secret
3. App Registration → API permissions
4. Teams app manifest → botId
5. Teams Admin Center → Manage apps / policies

That will be the real final fix for your reply issue.

If you want, after you test, just paste the output and say:

“done, now next”

System check identified no issues (0 silenced).
March 19, 2026 - 12:33:32
Django version 5.2.11, using settings 'wellnessreminder.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see:
https://docs.djangoproject.com/en/5.2/howto/deployment/

------ Teams Message Received ------
User Name: Sangram Bhosale
Teams User ID:
29:1QlyJZqWjHeDc_cj9iUvwYAG42zOPUIvBtiGI4d1KchVsOBzqJAPddTF_TVAMSEMH3cq_4HjnIA0Ci7CrX2tE-A
AAD Object ID: 16dd0967-ea3e-45b3-b457-22460a66ef2e
Conversation ID:
a:1ODVxDwJUuzE_tA_hIZ_rH7dF3VrK3JWKE8SBbhVhwh_xgBPczYi6i8_ofNuTAF5aDn71OsGoqhclAtvKCEVZ5n7PTA9yGSQ1psECnp5ATMGrPWOr-tpldVUL5GewYpUU
Tenant ID: 1d260f15-bd09-4056-ab2c-5e73843d7099
Service URL: https://smba.trafficmanager.net/in/1d260f15-bd09-4056-ab2c-5e73843d7099/
Message: Hi

```
------------------------------------

Stored Teams conversation details for Sangram
BOT SEND ERROR: Operation returned an invalid status code
'Unauthorized'
BOT MESSAGE ERROR: Operation returned an invalid status code
'Unauthorized'
Internal Server Error: /api/bot/messages/
[19/Mar/2026 12:33:46] "POST /api/bot/messages/ HTTP/1.1" 500 69

------ Teams Message Received ------
User Name: Sangram Bhosale
Teams User ID:
29:1QlyJZqWjHeDc_cj9iUvwYAG42zOPUIvBtiGI4d1KchVsOBzqJAPddTF_
TVAMSEMH3cq_4HjnIA0Ci7CrX2tE-A
AAD Object ID: 16dd0967-ea3e-45b3-b457-22460a66ef2e
Conversation ID:
a:1ODVxDwJUuzE_tA_hIZ_rH7dF3VrK3JWKE8SBbhVhwh_xgBPczYi6i8_of
NuTAF5aDn71OsGoqhclAtvKCEVZ5n7PTA9yGSQ1psECnp5ATMGrPWOr
-tpldVUL5GewYpUU
Tenant ID: 1d260f15-bd09-4056-ab2c-5e73843d7099
Service URL: https://smba.trafficmanager.net/in/1d260f15-bd09-4056-
ab2c-5e73843d7099/
Message: Hi
------------------------------------

Stored Teams conversation details for Sangram
BOT SEND ERROR: Operation returned an invalid status code
'Unauthorized'
```

BOT MESSAGE ERROR: Operation returned an invalid status code 'Unauthorized'
Internal Server Error: /api/bot/messages/
[19/Mar/2026 12:33:47] "POST /api/bot/messages/ HTTP/1.1" 500 69
i see this now when i message hi after above changes and still see those resonses and why i see it twice

Excellent — **this is exactly the milestone we wanted**.

Now your **code path is correct** and the problem is narrowed down cleanly.

✅ What this proves now

Your latest log proves:

Working correctly

- Teams → Azure Bot → ngrok → Django webhook ✅
- Bot activity parsing ✅
- AAD Object ID extraction ✅
- Employee mapping by `azure_user_id` ✅
- Teams conversation details saved in DB ✅

This line confirms it:

```
Stored Teams conversation details for Sangram
```

So your **programmatic foundation is now correct**. 🔥

❌ What is still failing

This is now the **only real blocker**:

```
BOT SEND ERROR: Operation returned an invalid status code 'Unauthorized'
```

That means:

> Your bot can **receive** Teams messages, but when it tries to **send reply back**, the Bot Framework connector rejects the bot's credentials.

So yes — this is now **Azure / Bot registration / Teams app configuration**, not Django logic.

🤔 Why do you see the message twice?

You asked a very good question.

You are seeing **two identical incoming requests** because **Teams / Bot Framework retries the delivery** when your endpoint returns **500 Internal Server Error**.

What is happening exactly

Flow right now:

1. You send `"Hi"` in Teams
2. Teams delivers activity to your bot endpoint
3. Your Django endpoint receives it

4. Your code tries:

```python
```

5. Bot Framework gets:
   - `Unauthorized`
6. Your view raises exception
7. Django returns:
   - `HTTP 500`
8. Since the bot endpoint returned failure, **Bot Framework / Teams retries the same activity**
9. So you see the same `"Hi"` again

That is why:

- same user
- same conversation ID
- same content
- near-identical timestamps
- appears twice

✅ **This is normal retry behavior when your endpoint returns 500.**

🎯 First small fix: stop returning 500 for send failures

Even if reply fails, you should **not crash the webhook**.

Because if webhook returns 500:

- Teams retries
- duplicate logs happen
- can create duplicate reminder actions later
- messy debugging

✅ Programmatic fix right now (very important)

We will make your bot endpoint:

- store Teams details
- try to reply
- if reply fails, **log it**
- but still return `200 OK`

This prevents duplicate retries.

STEP 1 — Update only `core/views.py`

📍 File path:

`core/views.py`

Inside `call_bot_logic()`, replace this block:

❌ Current block

```
if user_text:
    reply = f"Hello {user_name} 👋 I received: {user_text}"

    try:
        await turn_context.send_activity(reply)
        print("Reply sent successfully")
    except Exception as send_error:
        print("BOT SEND ERROR:", str(send_error))
        raise
```

✅ Replace with this

```
if user_text:
    reply = f"Hello {user_name} 👋 I received: {user_text}"

    try:
        await turn_context.send_activity(reply)
        print("Reply sent successfully")
    except Exception as send_error:
        print("BOT SEND ERROR:", str(send_error))
        # IMPORTANT: do NOT raise here
        # Otherwise Teams retries the same activity
```

STEP 2 — Also make outer exception safer

In the outer `except`, currently you return 500:

❌ Current

```python
except Exception as e:
    print("BOT MESSAGE ERROR:", str(e))
```

Python Run

✅ Replace with this

```python
except Exception as e:
    print("BOT MESSAGE ERROR:", str(e))
    # Return 200 to prevent Teams retry storms during debugging
```

Python Run

✅ Result after this change

Even if reply auth still fails:

- Teams will send only once
- no duplicate incoming `"Hi"`
- cleaner logs
- conversation details still saved

🔥 Now the REAL issue: `Unauthorized`

Now we move to **Azure-side root cause**.

This is almost always one of these:

1. `BOT_APP_ID` is wrong
2. `BOT_APP_PASSWORD` is wrong
3. Secret is expired / copied wrong (**Secret ID vs Secret Value**)
4. Azure Bot resource points to one App ID, but code uses another
5. Teams app manifest `botId` doesn't match Azure Bot App ID
6. Tenant / Teams admin has blocked the bot (`tenant admin disabled this bot`)

And from your screenshot, you **also definitely have**:

> **Microsoft Teams channel issue: "The tenant admin disabled this bot"**

So you likely have **two Azure-side problems at once**:

- **Bot auth mismatch** → causing Unauthorized reply
- **Teams governance policy** → causing channel issue

🚨 Very likely root cause in your case

Based on everything you shared, the **most likely culprit** is:

> You are using the wrong `BOT_APP_ID` and/or wrong `BOT_APP_PASSWORD`.

Because:

- Incoming request can still reach your webhook through Azure Bot Service
- But outgoing reply requires the bot to get a valid connector token
- Connector says **Unauthorized**
- That is classic **bot credential mismatch**

✅ Now I'll tell you EXACT Azure checks in the right order

Do these **exactly in this order**.

PHASE 1 — Verify Azure Bot credentials (MOST IMPORTANT)

STEP A — Open Azure Bot resource

Go to:

- **Azure Portal**
- open your bot resource:
  - `wellnessreminder-bot`

STEP B — Go to Configuration

In left panel:

- **Settings**
- **Configuration**

Look for:

- **Microsoft App ID**

Copy that exact value.

STEP C — Compare with your `.env`

In your `.env`, check:

```env
```

Rule:

This must be **exactly identical** to the **Microsoft App ID** shown in Azure Bot resource.

If even one character differs → reply will fail.

STEP D — Open the associated App Registration

From Azure Bot Configuration page, there is usually a link to the app registration / Manage password.

Or go manually:

- **Microsoft Entra ID**
- **App registrations**
- search by the **same Microsoft App ID**

Open that app.

STEP E — Create a NEW client secret (important)

Go to:

- **Certificates & secrets**
- **New client secret**
- create one

⚠️ **Copy the VALUE immediately**

Do **not** copy:

- Secret ID
- Description

You need:

✅ **Client secret VALUE**

STEP F — Update `.env`

Set:

```env
BOT_APP_ID=<same as Azure Bot Microsoft App ID>
```

⚠️ Very common mistake:

- using `GRAPH_CLIENT_SECRET` here by mistake
- using secret ID instead of secret value

STEP G — Restart Django completely

After changing `.env`:

```bash
CTRL + C
```

At startup, verify you still see:

```
BOT APP ID: ...
BOT PASSWORD LOADED: YES
BOT PASSWORD LENGTH: <non-zero>
```

PHASE 2 — Verify Teams app manifest

This is the second biggest cause.

STEP H — Open your Teams app `manifest.json`

In the Teams app package you uploaded/sideloaded, check:

```
"bots": [
  {
    "botId": "...."
  }
]
```

That `botId` must be:

✅ **exactly the same as Azure Bot Microsoft App ID**

STEP I — If you have `webApplicationInfo`

If your manifest contains:

```
"webApplicationInfo": {
  "id": "...",
  "resource": "..."
}
```

Then `id` should also typically be the same bot app ID if you're using that app identity.

STEP J — Rezip and re-upload the Teams app package

If you changed manifest:

- update zip package

- uninstall old bot from Teams
- upload the new package again

PHASE 3 — Verify Bot endpoint

STEP K — Update messaging endpoint

In Azure Bot → Configuration:

Set:

```
https://<your-current-ngrok-domain>/api/bot/messages
```

Because ngrok changes.

If ngrok URL changed and Azure still has old URL, weird behavior can happen.

PHASE 4 — Fix Teams admin policy issue (“tenant admin disabled this bot”)

Your screenshot clearly shows this issue exists.

Even after auth is fixed, this may still block/affect behavior.

STEP L — Open Teams Admin Center

Go to:

- **Teams Admin Center**
- **Teams apps**
- **Manage apps**

Search for your custom app / bot.

Check:

- It should be **Allowed**
- Not **Blocked**

If blocked → allow it.

STEP M — Check custom app upload policy

Go to:

- **Teams apps**
- **Setup policies** (or org-wide app settings depending on tenant UI)

Check for your user / global policy:

- **Upload custom apps** = **On**

If custom app upload is off, custom bot behavior can be blocked.

STEP N — Check permission policies / app-centric management

Depending on tenant UI:

- **Teams apps → Permission policies**

  or
- app-centric management in **Manage apps**

Ensure:

- custom apps are allowed
- or your app is explicitly assigned/allowed

STEP O — Check "The tenant admin disabled this bot"

That specific Azure channel issue usually means one of:

- app blocked in Teams Admin Center
- custom apps disabled
- app permission policy disallows it
- tenant governance blocking custom bots

So this must be fixed by tenant admin or by you if you have admin rights.

PHASE 5 — Check app registration permissions (important but secondary)

This is not usually the main cause of Unauthorized reply, but still check.

STEP P — Open App Registration → API permissions

Check what permissions are present.

For bot messaging, the Bot Framework handles connector auth; you do **not** need strange high-risk Teams permissions just to reply.

If you see:

- `Teamwork.Migrate.All`

👉 Remove it unless you truly need it.

There are reported cases where this causes Teams bot restrictions / admin disable behavior.

🧪 After you do the Azure fixes, what should happen?

When you send `"Hi"` again, you should see:

```
------ Teams Message Received ------
...
Stored Teams conversation details for Sangram
Reply sent successfully
```

And in Teams chat you should receive:

```
Hello Sangram Bhosale 👋 I received: Hi
```

That is the success signal.

✅ Do this tiny code fix now before Azure testing

Please make the **non-500** change first, so duplicate messages stop.

Final safe `core/views.py` change summary

1) In `call_bot_logic()`:

Do not `raise` after send error

2) In outer `except`:

Return **status=200** during debugging

📌 Why your bot still stores data correctly even though reply fails

This is important and good news:

Because you now see:

```
Stored Teams conversation details for Sangram
```

That means your DB already has:

- `teams_user_id`
- `teams_conversation_id`
- `teams_tenant_id`
- `teams_service_url`

So once the Azure auth is fixed:

- chat replies will work
- proactive reminders can work too

You've already completed the hardest code-side part.

🎯 What I want from you next (very specific)

Now send me **these exact 4 things** (no secrets):

1. Azure Bot → Configuration page values

Tell me:

- **Microsoft App ID**
- **Messaging endpoint**
- whether it says **Multi Tenant / Single Tenant / User Assigned MSI**

---

2. Your `.env` (only these lines, mask secrets)

Example format:

```
BOT_APP_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
BOT_APP_PASSWORD=********
GRAPH_CLIENT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
GRAPH_CLIENT_SECRET=********
GRAPH_TENANT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

3. Your Teams app `manifest.json` bot section

Just paste:

```
JSON
```

and if present:

```
JSON
```

---

4. Whether you have Teams Admin access

Just tell me:

- **yes, I am Teams admin**

  or
- **no, I need my admin**

🚀 Best next move

Reply with:

“I made the no-500 change, here are my 4 Azure details”

and I'll tell you **exactly which value is mismatched** and what to fix next.

i am still not getting any positive response
i want to make it work tell me how app id and secret is correct i didnt added any unwanted permission , programatically everything is good what is lacking

Yes — **now I can tell you exactly what is lacking**.

🔥 The missing piece is NOT your business logic anymore

Your programmatic logic for:

- receiving Teams messages
- storing conversation info
- mapping employee
- reminder logic

is **good enough now**.

The actual missing thing is:

❗ Your bot is almost certainly registered as Single Tenant, but your Python bot adapter is still using the old generic adapter config that behaves like multi-tenant/default auth.

That is why:

- inbound request reaches your endpoint ✅
- but outbound `send_activity()` fails with:
  - **Unauthorized** ❌

🎯 Exact diagnosis

You currently use:

`core/services/teams_bot.py`

```
from botbuilder.core import BotFrameworkAdapterSettings, BotFrameworkAdapter

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD
)

ADAPTER = BotFrameworkAdapter(SETTINGS)
```

This is the **old / minimal** pattern.

For many Teams bot setups — especially with **Single Tenant bot registrations** (which is the new normal after Microsoft's changes) — this is **not enough**.

✅ What is lacking programmatically

You are **not explicitly telling the SDK**:

- the bot is **Single Tenant**
- what the **tenant ID** is for bot authentication

That means the SDK may try auth against the wrong assumptions / wrong token flow, and the connector rejects reply tokens.

This aligns with Microsoft's newer single-tenant guidance and real-world failures after 2025 bot changes. In Microsoft Q&A discussions, developers report Teams deployment works only after using single-tenant settings including **app type + tenant ID**, while older auth assumptions cause `Unauthorized` on replies.

Microsoft Learn

✅ The real fix you should do NOW

We need to replace your old `BotFrameworkAdapterSettings` approach with the **newer authentication object approach**:

- `ConfigurationBotFrameworkAuthentication`
- `CloudAdapter`

This is the correct direction for modern Teams bots.

🚨 Important

Do NOT keep using `BotFrameworkAdapter` for this setup if your bot is single-tenant and giving reply Unauthorized.

Use:

- `CloudAdapter`
- `ConfigurationBotFrameworkAuthentication`

This is the modern and more reliable path.

✅ What we will change

We will update:

1. `core/services/teams_bot.py`
2. `core/views.py`
3. `core/services/teams_sender.py`

And add these env vars:

- `BOT_APP_TYPE=SingleTenant`
- `BOT_TENANT_ID=<your tenant id>`

STEP 1 — Update your `.env`

📍 File:

`.env`

Add these lines (or verify them):

```
BOT_APP_ID=your-bot-app-id
BOT_APP_PASSWORD=your-bot-client-secret-value
BOT_APP_TYPE=SingleTenant
BOT_TENANT_ID=your-tenant-id
```

Important:

- `BOT_APP_ID` = **Azure Bot resource Microsoft App ID**
- `BOT_APP_PASSWORD` = **Client Secret VALUE** of that same app registration
- `BOT_TENANT_ID` = your Entra tenant ID

STEP 2 — Replace `core/services/teams_bot.py`

📍 File:

`core/services/teams_bot.py`

❌ Delete old version

✅ Replace with this full code:

```python
import os
from dotenv import load_dotenv

from botbuilder.integration.aiohttp import ConfigurationBotFrameworkAuthentication
from botbuilder.core import CloudAdapter
from botframework.connector.auth import SimpleCredentialProvider

load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()
BOT_APP_TYPE = os.getenv("BOT_APP_TYPE", "SingleTenant").strip()
BOT_TENANT_ID = os.getenv("BOT_TENANT_ID", "").strip()

print("BOT APP ID:", BOT_APP_ID)
print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else "NO")
print("BOT PASSWORD LENGTH:", len(BOT_APP_PASSWORD) if BOT_APP_PASSWORD else 0)
print("BOT APP TYPE:", BOT_APP_TYPE)
print("BOT TENANT ID:", BOT_TENANT_ID)

# Build config dict for modern bot auth
BOT_CONFIG = {
    "MicrosoftAppType": BOT_APP_TYPE,           # SingleTenant
    "MicrosoftAppId": BOT_APP_ID,
    "MicrosoftAppPassword": BOT_APP_PASSWORD,
    "MicrosoftAppTenantId": BOT_TENANT_ID,      # REQUIRED for SingleTenant
}

AUTHENTICATION = ConfigurationBotFrameworkAuthentication(BOT_CONFIG)

ADAPTER = CloudAdapter(AUTHENTICATION)
```

⚠️ Very important note

If this import fails:

Python Run

that means your current package set may not include the integration package.

Then you need:

Bash

Add to `requirements.txt` too.

STEP 3 — Update `requirements.txt` if needed

📍 File:

`requirements.txt`

Add this line if not present:

```
botbuilder-integration-aiohttp==4.17.1
```

Then install:

Bash

STEP 4 — Update `core/views.py` for CloudAdapter

Because `CloudAdapter` uses a slightly different method than old `BotFrameworkAdapter`.

📍 File:

`core/views.py`

You currently use:

```
await ADAPTER.process_activity(
    activity,
    auth_header,
    call_bot_logic
)
```

That is old adapter style.

For `CloudAdapter`, use:

Python Run

OR in Django, since you already have deserialized activity, we can still use the request object style more cleanly by adapting less.
But easiest and most reliable for Django with current shape is:

Replace your entire `bot_messages` function with this:

```
@csrf_exempt
async def bot_messages(request):
    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"}, status=405)

    try:
        async def call_bot_logic(turn_context: TurnContext):
            user_text = (turn_context.activity.text or "").strip()

            user_name = turn_context.activity.from_property.name
            teams_user_id = turn_context.activity.from_property.id
            aad_object_id = getattr(turn_context.activity.from_property, "aad_object_id", None)

            conversation_id = turn_context.activity.conversation.id
            tenant_id = getattr(turn_context.activity.conversation, "tenant_id", None)
            service_url = turn_context.activity.service_url

            print("\n------ Teams Message Received ------")
            print("User Name:", user_name)
            print("Teams User ID:", teams_user_id)
            print("AAD Object ID:", aad_object_id)
            print("Conversation ID:", conversation_id)
            print("Tenant ID:", tenant_id)
            print("Service URL:", service_url)
```

```
            print("Message:", user_text)
            print("------------------------------------\n")

            employee = await get_employee_by_aad_object_id(aad_object_id)

            if employee:
                await store_employee_teams_details(
                    employee,
                    teams_user_id,
                    conversation_id,
                    tenant_id,
                    service_url
                )
                print(f"Stored Teams conversation details for {employee.display_name}")
            else:
                print(f"No employee matched with AAD Object ID: {aad_object_id}")

            if user_text:
                reply = f"Hello {user_name} 👋 I received: {user_text}"
                try:
                    await turn_context.send_activity(reply)
                    print("Reply sent successfully")
                except Exception as send_error:
                    print("BOT SEND ERROR:", str(send_error))
                    # Don't re-raise to avoid retries

        # CloudAdapter processes the raw request
        await ADAPTER.process(request, call_bot_logic)

        return JsonResponse({"status": "ok"}, status=200)

    except Exception as e:
        print("BOT MESSAGE ERROR:", str(e))
        return JsonResponse({"status": "received_with_error", "error": str(e)}, status=200)
```

⚠️ Important Django note

Depending on botbuilder version, `CloudAdapter.process()` may expect an **aiohttp request object**, not Django request.

If that happens, then we should instead use a **manual activity/auth wrapper** or keep old adapter and switch to explicit auth classes.

So if you get a method/signature error after this change, **don't panic** — I'll give you the Django-compatible fallback immediately.

But first, try the import + startup.

✅ Safer fallback (if CloudAdapter becomes troublesome in Django)

If you want the **least risky immediate fix**, there is another path:

- keep `BotFrameworkAdapter`
- but move to **single-tenant config in settings / env**
- and ensure correct app registration type + tenant ID + manifest

However, with Python SDK, the **modern adapter path is better** for this exact problem.

🚨 Another missing Azure-side thing that can still block you

Even with correct code, **if the app registration itself is not configured for Single Tenant correctly**, reply still fails.

You must verify in **Entra App Registration**:

Authentication / Supported account types:

Set to:

- **Accounts in this organizational directory only (Single tenant)**

And for bot auth, Microsoft guidance for single-tenant bots also requires using tenant-specific auth with `MicrosoftAppType=SingleTenant` and `MicrosoftAppTenantId`. This is the exact area where old configs commonly fail with `Unauthorized`. Microsoft Learn

---

🚨 One more Azure-side thing people miss

In **App Registration → Authentication**

Add this redirect URI if missing:

```
https://token.botframework.com/.auth/web/redirect
```

This is commonly required in bot auth setups and repeatedly mentioned in Microsoft guidance for single-tenant bot scenarios. Microsoft Learn

Even if you're not using OAuth prompt right now, I still recommend adding it for correctness.

❗ Most likely what is lacking in YOUR case

If I had to rank the likely causes **for your exact setup**, it is this order:

1. Missing single-tenant bot auth config in code

(**most likely**)

You are using old adapter style:

- no `MicrosoftAppType`
- no `MicrosoftAppTenantId`

2. Bot app registration is single-tenant but SDK is not told that

→ reply token acquisition fails

3. Teams app manifest `botId` mismatch

Still possible

4. Teams admin policy block

You already saw:

- **"The tenant admin disabled this bot"**

That must still be fixed too.

✅ My practical recommendation (fastest path to success)

First do this NOW:

Update `.env`

```env
BOT_APP_TYPE=SingleTenant
```

Then:

Update `core/services/teams_bot.py`

to the modern single-tenant auth version above.

🔍 What I need from you after that

After you do **just STEP 1 + STEP 2**, restart server and tell me:

1. Did import work?
   - or did it fail on `botbuilder.integration.aiohttp`
2. What startup logs show:
   - `BOT APP TYPE: ...`
   - `BOT TENANT ID: ...`
3. What happens when you send `"Hi"` again:
   - exact terminal output

🚀 Best next move

Reply with exactly one of these:

Option A

“I updated .env and teams_bot.py, here is the new error/output”

Option B

“Import failed for ConfigurationBotFrameworkAuthentication”

If you send that, I’ll give you the **exact next code** immediately (no guessing).

Sources

can we create new chatbot for it

Yes — and honestly, in your case, creating a fresh bot is a very good idea. ✅

Because right now your **backend logic is mostly fine**, but your current bot setup likely has **Azure Bot / App Registration / Teams app configuration drift** (mismatch somewhere), and that can waste hours.

So the **fastest clean path** may be:

✅ Create a brand new Teams bot from scratch

and connect it to your existing Django backend.

That gives you:

- clean **App ID**
- fresh **client secret**

- clean **Teams channel**
- clean **manifest**
- no hidden bad config from old bot
- easier debugging

🎯 My recommendation

Yes, create a new bot — but do it in a structured way.

Keep:

- your current Django codebase
- presence engine
- reminder logic
- employee DB
- proactive message storage logic

Replace:

- old Azure Bot resource
- old App Registration (if needed)
- old Teams app package/manifest

🚀 Best path: create a brand new clean bot and wire it correctly

I strongly recommend this exact approach:

OPTION A (Recommended): Create a brand-new Azure Bot + App Registration

This is the **best** path.

You will create:

1. **New Azure Bot resource**
2. **New App Registration** (or let Azure use/create one depending on flow)
3. **New client secret**
4. **New Teams app manifest**
5. Point it to your existing endpoint:
   - `https://<ngrok>/api/bot/messages`

Then update `.env`:

```env
BOT_APP_ID=<new bot app id>
BOT_APP_PASSWORD=<new bot secret value>
BOT_TENANT_ID=<your tenant id>
BOT_APP_TYPE=SingleTenant
```

⚠️ But important: don't create "another chatbot" in random way

You should **not** create:

- random Copilot agent
- Power Virtual Agents bot

- Microsoft 365 agent
- Teams “workflow bot”
- app-only Teams tab pretending to be bot

For **your use case** (Django backend + proactive reminders + Teams messaging), you need:

✅ A real Azure Bot / Bot Framework bot integrated with Teams

Because you need:

- incoming chat webhook
- outgoing replies
- proactive messages
- programmatic control from Django

🧠 For your use case, the correct architecture is:

Teams User

⬇️

Teams Channel

⬇️

Azure Bot Service

⬇️

Your Django endpoint (`/api/bot/messages`)

⬇️

Your presence/reminder logic

⬇️

Proactive message back via Bot Framework

That's exactly what you're building.

💯 My honest opinion

Since you've already spent time debugging the current bot and got:

- incoming works
- outgoing Unauthorized
- Teams channel issue says *tenant admin disabled this bot*

👉 **A fresh bot is probably faster than trying to rescue a corrupted/misaligned setup.**

✅ If you create a new bot, what stays unchanged in code?

Your current code can mostly stay.

You will mostly only change:

- `.env`
- maybe `teams_bot.py` depending on final adapter
- Teams manifest bot ID

Everything else can remain.

📌 What I would do if I were you

I would do this:

Step 1

Create **new App Registration** first (clean)

Step 2

Create **new Azure Bot resource** linked to that app

Step 3

Enable **Microsoft Teams channel**

Step 4

Set messaging endpoint:

```
https://<ngrok-url>/api/bot/messages
```

Step 5

Create new secret and put in `.env`

Step 6

Create a fresh Teams app manifest with:

- same new `botId`
- personal scope

Step 7

Upload fresh app to Teams

Step 8

Test `"hi"`

This is the cleanest route.

🔥 Important warning

Even if you create a new bot, if your **Teams admin policy** still blocks custom bots, you can still hit issues.

So if your tenant has:

- custom apps blocked
- bot blocked in Teams Admin Center
- app permission policy blocking it

then a new bot may still fail **unless** you fix that too.

So creating a new bot is **good**, but not a magic bullet if tenant policy is the root blocker.

✅ My recommendation: YES, create a new bot — and I can guide you exactly

If you want, I can now give you the **exact clean setup plan step by step**:

I will tell you in order:

1. Create **new App Registration** in Entra
2. Configure **supported account type**
3. Create **client secret**

4. Add **redirect URI**
5. Create **new Azure Bot resource**
6. Connect it to the App Registration
7. Enable **Teams channel**
8. Set **messaging endpoint**
9. Build the **correct Teams manifest.json**
10. Update your `.env`
11. Tell you the **minimum code changes**
12. Test chat reply
13. Then enable proactive reminders

🚀 Best next move

Reply with:

"yes, create new bot step by step"

and I'll give you the **exact full clean setup** in the correct order —
Azure Portal + Entra + Teams manifest + .env + minimal code changes — so we make this work properly this time.